Practical 8

Develop a mini AI application such as text classification, chatbot prototype or perception demo and report on ethics and explainability considerations.

Objective

To develop a mini AI application — a spam/ham text message classifier using a Naive Bayes model — and to report on its explainability (which words drive its predictions) and the ethical considerations involved in deploying such a classifier.

Procedure

1. Prepare a small labelled toy dataset of text messages tagged as 'spam' or 'ham' (legitimate).
2. Split the dataset into training and test sets using a stratified train/test split.
3. Convert the raw text into a bag-of-words feature representation using CountVectorizer.
4. Train a Multinomial Naive Bayes classifier on the vectorized training messages.
5. Evaluate the model on the held-out test set: report accuracy and a full precision/recall/F1 classification report.
6. Classify a set of new, unseen messages and report the predicted class and P(spam) confidence for each.
7. Extract and report the words most strongly associated with the 'spam' class from the model's learned log-probabilities (explainability).
8. Document ethics and explainability considerations for deploying this kind of classifier in practice.

Program / Code

```
# ================================================================
# PRACTICAL 8
# Mini AI Application - Spam / Ham Text Classification
# (with ethics and explainability reporting)
# ================================================================

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# ---------------- Toy Dataset ----------------
messages = [
    ("Win a free iPhone now, click this link!!!", "spam"),
    ("Congratulations, you have won a lottery prize", "spam"),
    ("Claim your free cash reward today", "spam"),
    ("Limited offer, buy now and get 50% discount", "spam"),
    ("You have been selected for a free gift card", "spam"),
    ("URGENT: your account will be suspended, verify now", "spam"),
```

```
    ("Hey, are we still meeting for lunch tomorrow?", "ham"),
    ("Can you send me the project report by evening?", "ham"),
    ("Don't forget to submit the assignment tonight", "ham"),
    ("Mom said dinner is ready, come home soon", "ham"),
    ("Let's catch up this weekend for coffee", "ham"),
    ("The meeting has been rescheduled to 3 PM", "ham"),
    ("Free entry to win a brand new car, act now", "spam"),
    ("Your invoice for this month is attached", "ham"),
    ("Congratulations! You are our lucky winner today", "spam"),
    ("Please review the attached document before Friday", "ham"),
]

texts = [m[0] for m in messages]
labels = [m[1] for m in messages]

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, random_state=42, stratify=labels
)

vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = MultinomialNB()
model.fit(X_train_vec, y_train)

y_pred = model.predict(X_test_vec)

print("=" * 65)
print("     MINI AI APPLICATION - SPAM / HAM TEXT CLASSIFICATION")
print("=" * 65)

print(f"\nTraining samples : {len(X_train)}")
print(f"Testing samples  : {len(X_test)}")

print("\nTest set predictions:")
for text, true, pred in zip(X_test, y_test, y_pred):
    mark = "OK" if true == pred else "WRONG"
    print(f"  [{mark:5}] true={true:<5} pred={pred:<5} | {text}")

print(f"\nAccuracy on test set : {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

# ---------------- New unseen messages ----------------
new_messages = [
    "Click here to claim your free vacation now",
    "Are you free to talk on the phone tonight?",
    "You won a prize, verify your bank details immediately",
    "Please find attached the notes from today's class",
]

print("-" * 65)
print("Classifying new / unseen messages:")
new_vec = vectorizer.transform(new_messages)
predictions = model.predict(new_vec)
probabilities = model.predict_proba(new_vec)
```

```
for msg, pred, prob in zip(new_messages, predictions, probabilities):
    spam_idx = list(model.classes_).index("spam")
    print(f"  '{msg}'")
    print(f"    -> Prediction: {pred}   (P(spam) = {prob[spam_idx]:.2f})")

# ---------------- Explainability: most informative words ----------------
print("-" * 65)
print("Explainability - words most associated with SPAM:")
feature_names = vectorizer.get_feature_names_out()
spam_idx = list(model.classes_).index("spam")
ham_idx = list(model.classes_).index("ham")
log_prob_diff = model.feature_log_prob_[spam_idx] - model.feature_log_prob_[ham_idx]
top_spam_words = sorted(zip(feature_names, log_prob_diff), key=lambda x: -x[1])[:8]
for word, score in top_spam_words:
    print(f"   {word:<12} weight={score:.2f}")

print("=" * 65)
```

Output

```
=================================================================
     MINI AI APPLICATION - SPAM / HAM TEXT CLASSIFICATION
=================================================================

Training samples : 12
Testing samples  : 4

Test set predictions:
  [OK   ] true=ham   pred=ham   | Mom said dinner is ready, come home soon
  [OK   ] true=spam  pred=spam  | Congratulations, you have won a lottery prize
  [WRONG] true=ham   pred=spam  | Your invoice for this month is attached
  [OK   ] true=spam  pred=spam  | Free entry to win a brand new car, act now

Accuracy on test set : 0.75

Classification Report:
              precision    recall  f1-score   support

         ham       1.00      0.50      0.67         2
        spam       0.67      1.00      0.80         2

    accuracy                           0.75         4
   macro avg       0.83      0.75      0.73         4
weighted avg       0.83      0.75      0.73         4

-----------------------------------------------------------------
Classifying new / unseen messages:
  'Click here to claim your free vacation now'
    -> Prediction: spam   (P(spam) = 0.99)
  'Are you free to talk on the phone tonight?'
    -> Prediction: ham   (P(spam) = 0.17)
  'You won a prize, verify your bank details immediately'
    -> Prediction: spam   (P(spam) = 0.90)
  'Please find attached the notes from today's class'
```

```
     -> Prediction: ham   (P(spam) = 0.13)
------------------------------------------------------------
Explainability - words most associated with SPAM:
   free          weight=1.39
   now           weight=1.39
   today         weight=1.11
   your          weight=1.11
   50            weight=0.70
   account       weight=0.70
   and           weight=0.70
   be            weight=0.70
============================================================
```

Conclusion

A mini spam/ham text classification application was built with a Naive Bayes model, reaching 75% accuracy on a small held-out test set and correctly flagging clearly spam-like unseen messages (e.g., 'Click here to claim your free vacation now' at 99% P(spam)) while correctly passing genuine messages. One test message ('Your invoice for this month is attached') was misclassified as spam, illustrating the real-world risk of false positives with a tiny training set. The explainability analysis showed words like 'free', 'now', 'today' and 'your' as the strongest spam indicators, which is intuitive and lets a user audit why the model made a decision. Ethically, such a classifier must be used with caution: with only 12 training examples the model is not production-ready and could disproportionately flag legitimate messages that happen to use urgent/promotional language (a false-positive/fairness concern), it offers no guarantee against adversarial phrasing designed to evade the learned word cues, and any deployment should keep a human-in-the-loop review step, report confidence scores (not just hard labels) as done above, and be retrained/audited periodically on a much larger and more representative dataset before being trusted for consequential decisions like blocking a message outright.